Faustus

Faustus is a Rust workspace for recording and inspecting Path of Exile market data. The first implementation slice contains GGG exchange ingestion, poe.ninja currency/fragment snapshots, SQLite storage, a local read-only service, and a GPUI desktop/WASM frontend.

Run Locally

Start the backend and Windows desktop app together:

```
.\scripts\dev.ps1
```

The script starts the loopback backend, waits for its health endpoint, then launches GPUI in the foreground. Closing the desktop app stops the backend. Service logs are written under `$env:TEMP\Faustus`.

The GPUI client loads the poe.ninja economy leagues, defaults to the first entry (the current temporary softcore trade league), and tells the service which league to watch. The service validates that selection and starts collecting immediately. Changing the dropdown switches the collector to the newly selected league. League options are cached by the service for 15 minutes.

To run the backend separately:

```
cargo run -p market-service -- serve
```

To trigger a single collection manually instead:

```
cargo run -p market-service -- ingest-once --league "LEAGUE_ID"
```

Each `ingest-once` run uses the same provider clients and cache validators as the scheduled collector.

When running the backend separately, launch the Windows GPUI frontend in another terminal:

```
cargo run -p market-ui --bin faustus-desktop
```

The frontend reads the service at `http://127.0.0.1:8787`. Its Current prices tab shows poe.ninja's current exchange estimates for the selected league, quoted in the overview's primary currency (Chaos for Standard), with volume, the dominant paired currency, trend, and fetch time. These are aggregate estimates, not executable buy/sell orders. The GGG history tab shows recorded pair snapshots and raw ratios separately.

The Gem flipping page compares poe.ninja SkillGem variants by level, quality, and corruption. Its shared search filters all four strategies, and clicking a gem row carries that gem into the search when switching strategies. Choose a start date from the available seven-point price history; the gem page currently ends at Now, while the shared range model supports selecting both endpoints for future analyses. It estimates level-1-to-max flips, 20% quality upgrades using 20 Gemcutter's Prisms, and the conditional profit of corrupted +1 level or +3% quality outcomes using prices at the selected endpoints. Facetor's Lens is an optional cost in Level Flip with an editable Chaos price and lens count. Variants with fewer than five poe.ninja samples or listings are marked low confidence; the checkbox includes or filters them. Lens XP varies and is not included in the gem feed. Estimates exclude trade fees, leveling time, failed corruptions, and market slippage.

The service also exposes:

- `GET /health`
- `GET /api/v1/leagues` (cached poe.ninja economy league options)
- `POST /api/v1/selection` with `{"id":"<league-id>"}`
- `GET /api/v1/status`
- `GET /api/v1/exchange/prices?league=LEAGUE_ID&type=Currency` (also supports `Fragment`; responses are cached for 15 minutes and revalidated with ETags)
- `GET /api/v1/gems?league=LEAGUE_ID` (poe.ninja SkillGem variants, prices, listing counts, and sparkline history; cached for five minutes)
- `GET /api/v1/exchange/markets?league=LEAGUE_NAME&limit=100`

The market query returns the latest stored snapshot for each pair in each league. The raw ratio and volume fields remain keyed by item identifier; the app does not infer a buy/sell price from them yet.

Data Collection Notes

GGG's documented exchange endpoint is a historical cursor feed. A request without a cursor starts at the oldest retained history, not at the current hour. The collector advances sequentially through at most 30 pages per batch, waits one second between page requests and one minute between incomplete batches, then polls hourly once caught up. It honors `Retry-After` after rate limits and does not issue undocumented cursor jumps. Current-hour exchange data and individual transactions are not exposed by this feed; a new database can take time to reach current-league rows.

poe.ninja's documented economy API provides Currency and Fragment stash overviews for stored snapshots, plus current exchange overviews for the prices tab. Current exchange responses are held in a per-league/category 15-minute service cache and revalidated with ETags. Requests go through the service, not directly from the frontend.

The browser UI currently has a compile-checked WASM target. Static asset packaging, hosting, browser interaction testing, and screenshots have not yet been added. For a same-origin deployment, serve the WASM app and API behind the same HTTPS origin.

GGG's API and poe.ninja's economy API can change without notice. Respect their rate limits and caching guidance. poe.ninja permits use of its documented economy endpoints but prohibits directly replicating its site; this app is intended for cross-source market analysis. Do not use undocumented site endpoints.

This product isn't affiliated with or endorsed by Grinding Gear Games in any way.

Workspace

- `market-core`: shared identifiers, exact decimal quotes, comparisons, and API types.
- `faustus-fetch`: GGG cursor-page client and raw response preservation.
- `ninja-fetch`: documented economy client, ETag handling, and overview DTOs.
- `market-store`: SQLite migrations, transactional cursor recording, and snapshot queries.
- `market-service`: one-shot ingestion CLI and local read-only HTTP API.
- `market-ui`: GPUI desktop and single-threaded WebAssembly frontends; major pages live under `src/pages/`.

GPUI is pinned to a Zed repository revision because its platform backend crates are not published on crates.io. The framework is pre-1.0; dependency updates may require source changes.